Автономная некоммерческая организация высшего образования
«Университет Иннополис»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)
по направлению подготовки
09.03.01 - «Информатика и вычислительная техника»

GRADUATION THESIS
(BACHELOR'S GRADUATION THESIS)
Field of Study
09.03.01 – «Computer Science»

Направленность (профиль) образовательной программы
«Информатика и вычислительная техника»
Area of Specialization / Academic Program Title:
«Computer Science»

Тема / Topic	**The implementation of a language server for the Jasmin cryptographic primitive description language / Реализация языкового сервера для языка описания криптографических примитивов Jasmin**

Работу выполнил / Thesis is executed by	**Бучнев Александр Андреевич / Aleksandr Andreevich Buchnev**	подпись / signature
Руководитель выпускной квалификационной работы / Supervisor of Graduation Thesis	**Кудасов Николай Дмитриевич / Nikolai Dmitrievich Kudasov**	подпись / signature

Иннополис, Innopolis, 2025

Contents

List of Tables

List of Figures

Abstract

Software engineers create programming languages for specific purposes, and effective usage of the programming language requires effective editing support. Announced in 2016, language server protocol aims to help software engineers obtain enriched support in their text editor (vim, neovim) or IDE (VSCode, Eclipse IDE) of choice by decoupling the 'language' and the 'editing' parts. Since modern IDEs and some text editors support language server protocol 'out of the box', it suffices for engineers to implement only one piece of software called language server to gain some editing features. For some reasons, not all software languages get language server support. One of these languages is Jasmin. Jasmin is a cryptographic primitive description language and the primary users of this language are the cryptography researchers. In this work, I present an implementation of a language server for the Jasmin programming language, which supports diagnostics, goto definition, incremental parsing, symbol highlighting, syntax highlighting, and symbol type on hover.

Chapter 1

Introduction

1.1 Language Server Protocol

Language server protocol announced in 2016 [1], and it allows software engineers to implement only one piece of software that can interact with several IDEs and text editors solving the $M \times N$ problem, where M is the number of programming languages that require 'language intelligence' support and N is the number of text editors that can provide such support.

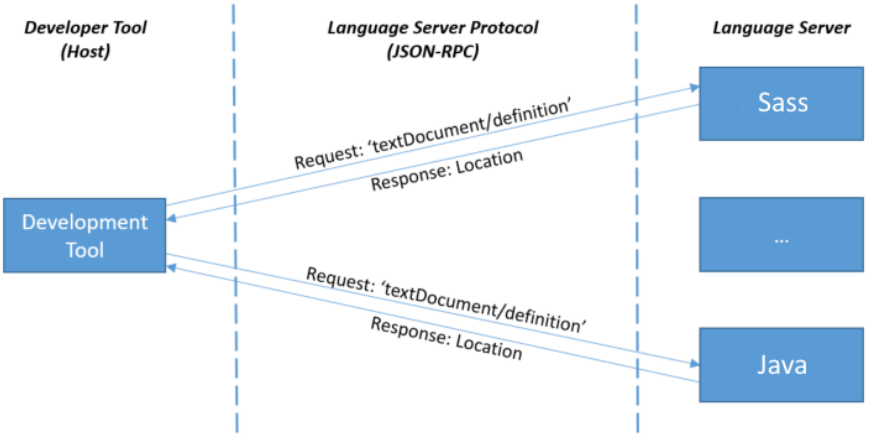

Fig. 1.1. The schematic represents interaction between IDE and different language servers [1]

The protocol itself consists of JSON-RPC 2.0 [2] messages representing different kinds of requests and notifications from both client and server. It is not necessary for a language server developert to support all features defined in the protocol. For example, one can only implement the type ascription (textDocument/hover) and 'Goto definition' (textDocument/definition) capabilities. Below is an example of communication between language server and the client. The protocol itself does not define a specific transport that the developers must use to communicate with the server. The communication might be implemented through the API calls, using IPC (Inter Process Communication) as a separate running binaries or even through the network.

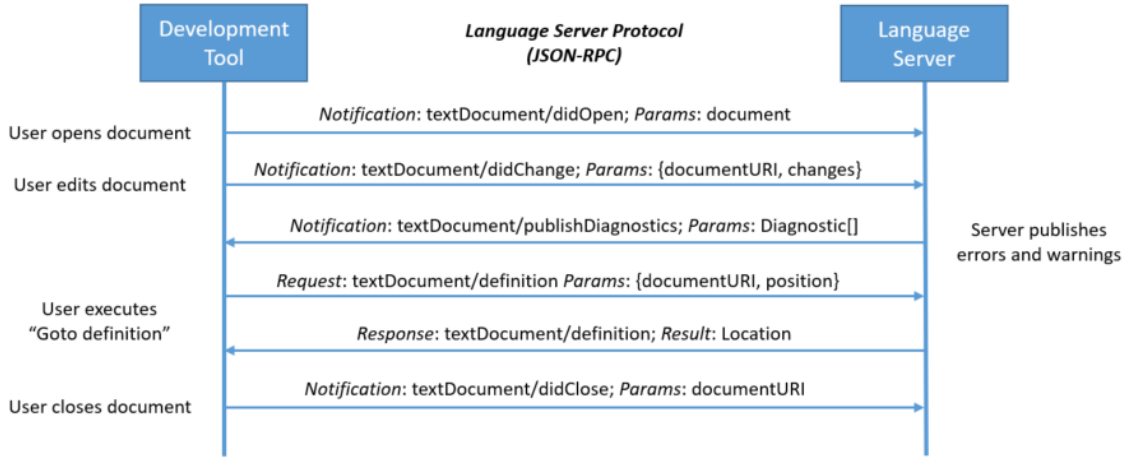

Fig. 1.2. Routine communication between language server and development tool [1]

1.2 Jasmin programming language

Cryptographic software plays a crucial role in modern software development. Although it typically constitutes only a small part of the overall codebase, it often covers the most critical security functions, e.g. the OpenSSL library implements the SSL/TLS protocols, which are fundamental to securing the vast majority of

internet communications.

Serious vulnerabilities in cryptographic software show how crritical it is. For example, the Heartbleed [3] vulnerability (CVE-2014-0160), which was publicly disclosed in 2014. This flaw affected systems running vulnerable versions of OpenSSL, allowing attackers to read server memory. As a result, private keys used for traffic encryption could be exposed, allowing an attacker to access unencrypted HTTPS traffic.

To lower the risks of implementation vulnerabilities, researchers have developed programming frameworks that enable formal verification of cryptographic primitives against attacks. One such framework is Jasmin [4].

Jsamin is a cryptographic framework designed for implementing cryptographic primitives, such as hash functions or digital signature schemes. One of the recent studies [5] showed its applicability for modern cryptographic primitives.

Additionally, Jasmin compiler allows for the transpilation of Jasmin source code into EasyCrypt [6] specifications for further formal verification of security properties.

This work focuses primarily on the Jasmin language itself, particularly, the language server implementation for the Jasmin language.

1.3 Contribution

In this work, we report the progress in implementing a language server infrastructure for the Jasmin programming language.

The principal contributions of this research include the implementation of a language server for the Jasmin programming language, which has:

- Diagnostics support for the Jasmin language

- Goto definition support (textDocument/definition)
- Reference highlight support (textDocument/highlight)
- Type ascription support (textDocument/hover)
- Tree-sitter support for the Jasmin language

Chapter 2

Literature Review

This work is based on previous researches in the field of computer science, including ones on language server and compiler construction. In this chapter we conduct the literature review of the most relevant sources in these areas.

2.1 Language Server Protocol

The Language Server Protocol (LSP) is a language-agnostic protocol designed to ease the integration of different programming languages with development environments. Before the introduction of LSP, language developers had to implement and maintain distinct integration for each supported IDE, which was both time-consuming and resource-intensive. LSP offers a unified protocol that could be implemented once per language and used across multiple editors.

The current specification of LSP, version 3.17 [1], defines a standard suite of core editing features, such as ‘go to definition’, ‘symbol renaming’, ‘code completion’, and ‘semantic highlighting’. These features improve the development experience by providing essential tools for code navigation.

It is worth noting, that, LSP was designed as an extensible protocol, allowing language developers to implement additional functionalities via custom requests and responses.

As an example of such custom functionality, the OCaml language server [7] introduces a custom request method, $\mathrm{ocamllsp/switchImplIntf}$, which allowes the developer to toggle between interface ($\mathrm{foo.mli}$) and implementation ($\mathrm{foo.ml}$) files. This feature highlights the flexibility of LSP to accommodate language-specific requirements.

2.1.1 Language server implementations

For optimal integration with the Jasmin compiler, which is implemented in OCaml, the language server was developed in OCaml too. This choice provides direct access to compiler API for essential operations, including source code parsing and functions on typed AST. Below, we analyze relevant OCaml implementations to derive best practices for language server design.

Jasmin language server

Aside from the implementation discussed in this work, the Jasmin Language Server described in [8] is the only existing option for providing basic diagnostics support for Jasmin. While functional, this implementation has several limitations:

- It uses a custom-written Language Server Protocol (LSP) wrapper. This choice complicates maintenance and extensibility, especially given the availability of community-supported LSP libraries for OCaml, such as [7].
- Logging is performed directly to stderr, which is non-extensible and limits the ability to effectively debug and trace issues. A more robust logging

framework would facilitate better development and maintenance processes.

- The server currently supports only the $\mathrm{textDocument/publishDiagnostics}$ functionality. It omits other essential LSP features such as hover information, go-to definition, and find references, which are critical for improving the user experience and productivity.

- The server lacks configurability, meaning it cannot parameterize settings such as the include path or message severity levels. This restriction may limit its applicability in some projects.

OCaml language server (Merlin)

Another example in the field of language servers is Merlin, a sophisticated tool specifically designed for the OCaml programming language. Merlin's development started in 2013, long before the introduction of the Language Server Protocol (LSP). In their report [9], the developers and researchers behind Merlin share valuable insights and methodologies that were instrumental in crafting this language server [10].

One of the major contributions of the Merlin project is its sophisticated management of incomplete code during real-time editing, a feature that is essential to providing precise diagnostics and stable code suggestions, even when the code is incomplete. This ability is crucial for developers as it supports a seamless programming experience by detecting errors at the earliest, providing real-time autocomplete suggestions. Becoming able to do this is is challenging and needs a powerful parsing technique and strong type inference mechanisms for developing well-informed assumptions of the programmer's intentions, despite incomplete input.

Moreover, Merlin's architecture shows the potential to integrate deeply with IDEs, offering features like type checking, code navigation, and more. These capabilities not only enhance the development experience but also set a high standard for language servers in terms of user support.

Semgrep language server

Particularly interesting language server implementation is the Semgrep Language Server [11], which served as a primary technical reference due to its clarity and modular design.

While the implementation supports only the core LSP features, it presents pragmatic architectural choices that help maintaining efficiency and simplicity of a language server. For example, it uses the ocaml-lsp library [7] for LSP messages handling, which simplifies communication between the editor and the server by abstracting away the complexities of the LSP implementation. Furthermore, the Semgrep Language Server utilizes the lwt library [12] for asynchronous input/output handling, enabling non-blocking operations that can be run in a separate thread.

Additionally, the modular design of Semgrep language server helps with new feature addition, due to the fact that most of Semgrep language server internal functions are implemented as pure functions, that greatly helps with debugging and testing new features.

2.1.2 Libraries for language server development

A widely adopted OCaml library for language server development is the aforementioned ocaml-lsp library [7]. As of this writing, this library provides

comprehensive support for processing JSON-RPC 2.0 [2] communication, which is the underlying protocol for the Language Server Protocol (LSP).

One of the key features of the ocaml-lsp library is its capability to handle bidirectional conversion between raw protocol messages and typed OCaml representations of LSP structures. This allows developers to work with strongly-typed data, reducing the likelihood of runtime errors.

Internally, the library relies on the yojson library [13] for efficient JSON parsing and serialization. yojson is well-suited for this task due to its performance and ease of use, enabling rapid conversion between JSON data and OCaml types.

The architecture of ocaml-lsp enables developers to concentrate on implementing language-specific logic, such as syntax analysis, code completion, and other higher-order functionalities, without delving into the low-level details of protocol handling. Furthermore, the library's open-source nature invites community contributions, leading to continuous improvements and feature enhancements.

2.2 Jasmin Compiler

The Jasmin compiler [4] is a formally verified toolchain specifically designed for the compilation of high-assurance cryptographic code into efficient and secure assembly. One of its primary features is the ability to verify program properties such as memory safety and termination through its integrated static analyzer. This ensures that the generated assembly code is not only performant but also compliant to security standards, making it highly suitable for cryptographic applications. However, utilizing the full range of the compiler's utilities necessitates a deep understanding of its source code, as there is no public API provided for more straightforward interaction.

A distinctive attribute of the Jasmin compiler is its capability to extract programs to EasyCrypt [6] specifications. This feature is particularly significant for advancing the formal verification of cryptographic algorithms, as it allows developers to conduct thorough correctness and security proofs within the EasyCrypt framework. The authors assert that for a Jasmin program that meets safety criteria, its corresponding EasyCrypt embedding remains sound. This claim underscores the integrated approach of Jasmin and EasyCrypt in ensuring the verification of cryptographic software, highlighting the importance of formal methods in the development of secure and verified systems. Such a combination not only enhances the assurance of security properties but also facilitates ongoing formal analysis and proofs of modern cryptographic algorithms.

2.3 Tree-sitter

Tree-sitter is a powerful and flexible parser generator tool designed to build fast and robust parsers that produce concrete syntax trees (CST) for use in code editors and other tools. Developed by Max Brunsfeld and first released in 2018, Tree-sitter has quickly gained popularity due to its efficiency, ease of use, and comprehensive support for a wide variety of programming languages [14].

One of the primary advantages of Tree-sitter over traditional parsing techniques is its incremental parsing capability, which allows parsers to update the syntax tree efficiently in response to code changes. This feature is particularly beneficial in the context of text editors and Integrated Development Environments. Incremental parsing reduces the computational overhead associated with re-parsing the entire file, enabling smoother and more intuitive coding experiences for developers.

Chapter 3

Methodology

This chapter describes the methodology used to implement language server for the Jasmin programming language. The primary goal is to build a working language server prototype and develop the integration for one of the text editors that support LSP for further extensions and improvements. The following sections provide the requirements, general server architecture, testing methodology and other design considerations.

3.1 Requirements and Development approach

In this section we detail the requirements we pose onto the language server features.

3.1.1 Requirements

The design of the language server necessitates support for core language intelligence features, which are outlined below:

- Symbol hover functionality (type ascription)

- Go to definition
- Syntax/semantic highlight support
- Source code diagnostics (parsing, typechecking)

Also it is important to have not only the language server executable binary itself, but also clear instructions on how to use the language server in one's text editor of choice.

3.1.2 Design

Considering the OCaml and Semgrep language server design mentioned in 2.1.1, we have chosen to follow the same approach and implement the language server in a way that our implementation must be easily extendable and modifiable, allowing and process each client request asynchronously.

Judging by the requirements, the language server would be implemented in OCaml, since the Jasmin compiler is written in OCaml, allowing easy integration with the compiler API.

The Jasmin language server implementation can be divided into two core components: JSON-RPC Server and a group of LSP handlers. RPC server represents the communication bus between the LSP client and the language server.

Server handles low level protocol operations, including parsing raw JSON-RPC messages, converting these messages between JSON and OCaml internal representation, routing the messages to the corresponding handlers and error handling for malformed messages.

LSP handlers, in turn can be divided in two groups too: the request handlers and the notification handlers. Request handlers, as expected, process LSP requests,

e.g. textDocument/definition, textDocument/hover. It is worth noting that each request must be completed within a feasible timeout, also it is important to keep in mind during the implementation, that the ordering of the requests is important too. The server may reorder independent requests like textDocument/completion and textDocument/signatureHelp, as they don't affect each other's results. However, dependent requests like textDocument/definition and textDocument/rename must maintain order, since renaming could invalidate definition results. Notification handlers, in general, do not require immediate server response, in some cases (inialized) notification does not require the server response at all. Both handler groups share the access to the compiler's API for semantic analysis, and the shared server state.

To implement syntax or semantic highlight support for the code, one might implement the corresponding LSP capability, which will collect all semantic tokens of the source code tree and then map these tokens to the LSP tokens. In our case we are going to implement the tree-sitter for the syntax highlight instead, getting the syntax highlight support only.

For more information on tree-sitter and its usage in this work please refer to Section 4.3.1.

3.1.3 Development approach

Instead of releasing the whole language server implementation at once, we chose to follow iterative development approach, splitting the development process into stages. The main development stages outlined below.

Implementing syntax highlight support

First, we implement the syntax highlight support utilizing existing Jasmin grammar definition. We transform the BNF-like grammar for Menhir [15] to tree-sitter Javascript, and then test the obtained grammar on the corpus of correct (in terms of grammar) Jasmin files. These files can be found in the libjade [16] repository.

Implementing core LS initialization routine

We implement the JSON-RPC event loop handler, which processes raw JSON-RPC messages (e.g., $\mathrm{initialize}$, $\mathrm{textDocument/didOpen}$ capabilities). This includes parsing and validating incoming requests and responses and managing the Language Server state.

Adding diagnostics support

We integrate the compiler's API to parse the source into the CST, type-check the CST and transform it into the AST, map compiler diagnostics to the LSP-compliant diagnostics and support real-time updates, e.g. on file changes.

Adding code intelligence and documentation

We proceed with further extension of language server capabilities by extending AST interaction to provide symbol information (e.g., definitions, references). We implement functionality to get the symbol type on cursor hover, as well as write clear documentation on how to use the language server.

3.2 Testing methodology

Since our implementation relies on the compiler-provided parser and type-checker as an external library, we assume their correctness and do not test their core functionality.

For the remaining components, we employ the following testing strategies:

- **Unit Testing.** We verify the correctness of AST transformations, code generation, and other critical logic. These tests ensure proper interaction with the compiler's API (e.g., correct function calls, error handling).
- **Manual Testing** We perform hands-on verification by running the compiled binary and observing its behavior with an LSP client (Neovim).

Chapter 4

Implementation

4.1 Jasmin language server

This section presents the implementation of a Language Server Protocol (LSP) interface for the Jasmin programming language, developed in OCaml. The server directly integrates with Jasmin's compiler frontend, utilizing its existing type system and AST representation to provide language features through LSP communication.

A reference Neovim client configuration demonstrates practical integration, showcasing real-time static analysis and editor tooling capabilities while maintaining consistency with the compiler's behavior.

4.1.1 Implementation details

The language server's core functionality is implemented in pure functions where possible in the request handler components. This design choice significantly simplifies both debugging and future extensions to the system. OCaml's advanced type system provides benefits in this implementation, enabling compile-

```
1  let start (server : Rpc_server.t) =
2    let rec rpc_loop server =
3      match server.state with
4      | State.Stopped ->
5        Logs.info (fun m -> m "Server stopped");
6        Lwt.return_unit;
7      | _ -> (
8        let%lwt client_msg = Utils.read () in
9        match client_msg with
10       | Some msg -> (
11         let%lwt server =
12           let server, reply_result =
13           Message_handler.handle_message server msg
14         in Lwt.dont_wait
15           (fun () -> Utils.send reply_result)
16           (notify_and_log_error_sync "Error");
17         Lwt.return server
18       in rpc_loop server
19       )
20       | None ->
21         Logs.debug (fun m -> m "Client disconnected");
22         Lwt.return_unit
23     )
24     in rpc_loop server
```

Listing 4.1: Main server loop

time verification of complex invariants.

Let us proceed with the RPC server. The RPC server architecture employs a state machine model with three distinct states: *Uninitialized*, *Running*, and *Stopped*, as presented in Listing 4.1.

In the *Uninitialized* state, the server process has started but has not yet established any client connections. The transition to the *Running* state occurs when processing the client's initialize request, which triggers several critical initialization procedures. These include setting up the document storage system (implemented as an in-memory key-value store tracking open files) and preparing various language service components, ending the initialization process with the server response containing supported capabilities.

Listing 4.4 features the part of the initialization routine, which at the end creates (Listing 4.4, line 3) the capabilities variable representing language server

supported capabilities with some modifiers.

The *Running* state maintains strict operational constraints, with only one possible state transition — to *Stopped*. This transition occurs under two conditions: either when encountering unrecoverable errors (e.g. broken communication pipe) or when processing a client `shutdown` request, which gracefully terminates the language server.

Main server loop is implemented as the recursive call to the `rpc_loop` function (Listing 4.1, line 2). During the loop execution, the function matches the server state, and if the server is stopped, it does not process client messages. In other case (Listing 4.1, line 7), it starts client message handling in a separate `lwt` thread, so the main thread can continue looping. This is possible, since LSP designed in a way, that allows client to make requests without waiting for server to respond before making a new one.

When the RPC server accepts the client JSON-RPC message, the server passes message to the corresponding message handler, as shown in Listing 4.2. On line 3 starts the pattern matching on the received JSON-RPC message, and during this matching, the server can process either LSP request, or LSP notification. In case (on line 13) some other LSP message was sent to the language server, nothing happens.

Logging is performed into the separate configurable file, which allows for more convenient debugging. This implementation divides all logging messages into three categories: debug, informational and error. Listing 4.3 contains function which creates a file on the host file system with correct permissions and sets up logging reporter with debug log level.

Our language server implementation is designed to run as a standalone binary which communicates with the client via process stdin and stdout.

```
let handle_message (server : Rpc_server.t) (msg : Packet.t) =
  Logs.debug (fun m -> m "handling message");
  match msg with
  | Request req -> (
    Logs.debug (fun m -> m "handling request");
    on_request server req
  )
  | Notification notification -> (
    Logs.debug (fun m -> m "handling notification");
    let server, reply = handle_notification server notification in
    in server, reply
  )
  | _ -> (
    Logs.debug (fun m -> m "unknown message");
    server, msg
  )
```

Listing 4.2: Message handlers

```
let setup_logging_to_file filename =
  let file_formatter =
    let oc = open_out_gen [Open_append;Open_trunc;Open_creat] 0o600 filename
        in
    Format.formatter_of_out_channel oc
  in

  let reporter = Logs_fmt.reporter ~dst:file_formatter () in

  Logs.set_reporter reporter;
  Logs.set_level (Some Logs.Debug);
  ()
```

Listing 4.3: Logging set up function

4.1.2 Features

Currently, language server supports the features specified below.

Real-time code diagnostics

Real-time code diagnostics is a vital feature in software development environments, as it enables programmers to identify type and syntax errors in the editing phase without executing the compiler.

In our implementation, the code diagnostics process is executed within a separate thread, triggered whenever the user saves the file they are working on, processing the entire modified content.

As illustrated in Listing 4.4, specifically on line 9, our language server supports the `Full` synchronization kind, meaning that when the user saves or makes changes in the file, on each change, the whole contents of the file is sent to the language server. In ideal case, language server should accept the incremental changes, but this is not currently supported, and probably will require some modifications to the compiler's source code itself, since the compiler does not expose an API for incremental parsing.

The code diagnostics works in two distinct phases to provide comprehensive feedback to the user. In the first phase, the language server attempts to parse the source code each time it detects a change. Effective error notification is crucial at this stage, as it alerts the user when the language server encounters parsing errors. User notification implemented by sending a server notification request to the client, including all parsing errors encountered, as depicted in Figure 4.1, where the compiler could not parse the token `tooo` and returned an error.

The second phase involves type inference, a critical process that requires the language server to interface with the compiler API to obtain the typed AST. During

```
let on_request (server : Rpc_server.t) (_params : InitializeParams.t) =
  ...
  let capabilities =
    ServerCapabilities.create
    ~textDocumentSync:
      ( `TextDocumentSyncOptions
        (TextDocumentSyncOptions.create
          ~openClose:true
          ~change:TextDocumentSyncKind.Full
          ()
        )
      )
    ~workspace:
      (ServerCapabilities.create_workspace
         ~workspaceFolders:
           (WorkspaceFoldersServerCapabilities.create ~supported:true
              ~changeNotifications:(`Bool true) ())
         ())
    ~documentHighlightProvider:(`Bool true)
    ~definitionProvider:(`Bool true)
    ~hoverProvider:(`Bool true)
    ()
  in
  let serverInfo =
    InitializeResult.create_serverInfo ~name: "jasmin-lsp" ~version:"0.0.1" ()
        in
  let init =
    InitializeResult.create ~capabilities ~serverInfo () in
  let server =
    {
      internal_state = new_internal_state;
      state = Rpc_server.State.Running
    }
  in server, init
```

Listing 4.4: Language server supported features

this phase, the compiler runs its internal typechecking algorithm that inferences all variable types and checks for illegal memory access Any errors detected at this stage, such as type mismatches, are also communicated to the user, as illustrated in Figure 4.2, where one tries to assign an array variable type ($\mathrm{stack\ u64[4]}$) to an array element, where elements are expected to be of type $\mathrm{u64}$.

Symbol highlight

The implementation of a symbol highlighting in our language server involves fetching additional info from the typed AST. Firstly, we locate relevant AST node in the internal representation that corresponds to the symbol under the cursor.

We then traverse the entire AST to identify all instances of that symbol with the same identifier. This can easily be done, since Jasmin compiler assigns unique identifiers to each symbol during the parsing and type inference stages. These identifiers remain distinct across different compiler invocations, ensuring that symbols are uniquely distinguished even if they share the same name.

For instance, if a programmer redeclares a variable with an identical name elsewhere in the code, the internal AST representation treats them as separate entities, each with its own unique identifier. An example of symbol highlighting, demonstrating this capability, is illustrated in Figure 4.3, where the variable with the name $\mathrm{x2}$ is highlighted in all its usages.

Go to definition

We implemented go to definition feature using the compiler's internal representation of the symbol in the AST. Each node contains not only it's location in the project, but also, the definition location if applicable, making the implementation

of this feature rather easy.

Symbol type inference on hover (Type ascription)

In developing the symbol type inference for our language server, we relied on the Jasmin compiler's API to extract detailed information from the typed AST. This API offers a set of functions for pretty printing, which enabled us to directly access and format each symbol's type information. The primary goal of implementing this LSP capability was to display the symbol's type to the user in a clear, concise manner without undertaking additional operations on the type data itself. By using the pretty-printing function, we could seamlessly present this information as a hover tooltip, improving the coding experience.

As illustrated in Figure 4.4, when the user hovers over the variable z, the hover feature accurately displays its type as reg u64.

4.2 Installation and configuration

An example of how can one configure Neovim text editor to work with Jasmin language server is presented in Listing 4.5. The configuration features the usage of filetype detection mechanism in Neovim, adding support for Jasmin (.jazz) files, as well as error handling logic, when the language server could not start for some reasons.

As it was mentioned earlier in section 4.1.1, our implementation is meant to run as a standalone binary, so, from the Listing 4.5 (line 9) one can see that the text editor process spawnes the language server proccess for further interaction.

```
vim.filetype.add {
  extension = {
    jazz = 'jazz',
  },
}

local client = vim.lsp.start_client {
  name = 'jasmin-lsp',
  cmd = { '/path/to/jasmin-lsp/binary' },
  on_attach = vim.notify 'client initialized',
}

if not client then
  vim.notify 'client error'
end

vim.api.nvim_create_autocmd('FileType', {
  pattern = 'jazz',
  callback = function()
    vim.lsp.buf_attach_client(0, client)
  end,
})
```

Listing 4.5: LS configuration example for neovim text editor

4.3 Tree-sitter support

4.3.1 Tree-sitter

Tree-sitter [14] is a parsing system that can generate incremental parsers for arbitrary user-defined grammars. It builds concrete syntax trees that update in real-time as the source code changes, making it particularly suitable for text editor tooling and language servers. Unlike traditional parsers, tree-sitter handles incomplete code gracefully, enabling syntax highlighting and code navigation.

4.3.2 Implementation

This section describes the implementation of the tree-sitter parser for the Jasmin programming language.

In early stage of the language server development, we have implemented

the tree-sitter parser for the Jasmin programming language, as it provided syntax highlighting, improving overall development experience.

As shown in Figure 4.5, on the right hand side is the concrete syntax tree (CST) for the code snippet, and the highlighted part on the left hand side is the corresponding CST node of the source code. From this code snippet it can be seen, that the highlight queries give the flexibility in what parts of the concrete syntax tree we want to highlight.

The development process for a Tree-sitter parser typically involves two critical phases: crafting a grammar for the target language and developing 'highlight queries' for the established grammar. Writing the grammar involves defining the syntactic structure of the language, ensuring that all valid constructs are accurately represented. This step is foundational, as it dictates how source code is parsed into a concrete syntax tree (CST).

The Jasmin compiler utilizes Menhir [15] as its parser generator tool for constructing the front-end component of the compiler. Menhir is a parser generator for OCaml, known for its capacity to handle large and complex grammars efficiently. It enables compiler developers to define the language's syntax in a Backus-Naur Form (BNF)-like syntax, which simplifies expressing the grammatical structure of programming languages. This high-level abstraction allows for more readable and maintainable grammar specifications.

Once the grammar is formulated, Menhir generates the corresponding lexer and parser from this definition. This automatic generation is crucial, as it abstracts the underlying complexity of syntax analysis and reduces human errors. Menhir's error error handling and support for advanced parsing techniques, such as LR(1), further enhance the compiler's ability to provide meaningful syntax error messages and efficient parsing routines.

```
local parser_config = require('nvim-treesitter.parsers').get_parser_configs()
parser_config.jasmin = {
  install_info = {
    url = 'https://github.com/y4cer/tree-sitter-jasmin',
    files = { 'src/parser.c' },
    generate_requires_npm = false,
    requires_generate_from_grammar = false,
  },
  filetype = 'jazz',
}
```

Listing 4.6: Tree-sitter configuration example for neovim text editor

Highlight queries are particularly important because Tree-sitter generates and utilizes the CST without knowledge of the semantics represented by each node. For example, the CST does not inherently distinguish between nodes representing function identifiers and those representing user-defined types. By implementing highlight queries, we can assign semantic meaning to specific CST nodes, providing color-coded syntax highlighting that enhances code readability and aids developers in quickly identifying code elements.

4.3.3 Installation and configuration

Assuming that the user has the Jasmin file type supported in Neovim, they can use the configuration presented in Listing 4.6 to add tree-sitter support for the Jasmin programming language.

This configuration adds the tree-sitter mapping for the jazz filetye, which represents the Jasmin language, specifying that the parser firstly must be downloaded from the external github repository.

After modifying the configuration user must install the tree-sitter for the Jasmin separately.

```
   20 inline fn __init_points4(
   19   reg u64[4] initr)
   18   ->
   17   stack u64[4],
   16   reg u64[4],
   15   stack u64[4],
   14   stack u64[4]
   13 {
   12   inline int i;
   11   stack u64[4] x2 x3 z3;
   10   reg u64[4] z2r;
    9   reg u64 z;
    8
    7   ?{}, z = #set0();
    6
    5   x2[0] = 1;
    4   z2r[0] = 0;
    3   x3 = #copy(initr);
    2   z3[0] = 1;
    1
E 21    for i=1 tooo 4    parse error: unexpected token
    1   { x2[i] = z;
    2     z2r[i] = z;
    3     z3[i] = z;
    4   }
    5
    6   //     (1,   0, init, 1)
    7   return x2, z2r, x3,  z3;
    8 }
```

Fig. 4.1. Example of parsing error

```
1   inline fn __init_points4(
1     reg u64[4] initr)
2     ->
3     stack u64[4],
4     reg u64[4],
5     stack u64[4],
6     stack u64[4]
7   {
8     inline int i;
9     stack u64[4] x2 x3 z3;
10    reg u64[4] z2r;
11    reg u64 z;
12
13    ?{}, z = #set0();
14
15    x2[0] = 1;
16    z2r[0] = 0;
17    x3 = #copy(initr);
18    z3[0] = 1;
19
20    for i=1 to 4
21    { x2[i] = z;
22      z2r[i] = z;
E 23    z3[i] = z3;    ■typing error: can not implicitly cast u64[4] into u64
24    }
25
26    //     (1,   0, init, 1)
27    return x2, z2r, x3,  z3;
28  }
```

Fig. 4.2. Example of typing error

```
15 inline fn __init_points4(
14   reg u64[4] initr)
13   ->
12   stack u64[4],
11   reg u64[4],
10   stack u64[4],
 9   stack u64[4]
 8 {
 7   inline int i;
 6   stack u64[4] x2 x3 z3;
 5   reg u64[4] z2r;
 4   reg u64 z;
 3
 2   ?{}, z = #set0();
 1
16   x2[0] = 1;
 1   z2r[0] = 0;
 2   x3 = #copy(initr);
 3   z3[0] = 1;
 4
 5   for i=1 to 4
 6   { x2[i] = z;
 7     z2r[i] = z;
 8     z3[i] = z;
 9   }
10
11   //     (1,   0, init, 1)
12   return x2, z2r, x3,  z3;
13 }
```

Fig. 4.3. Example of symbol highlight

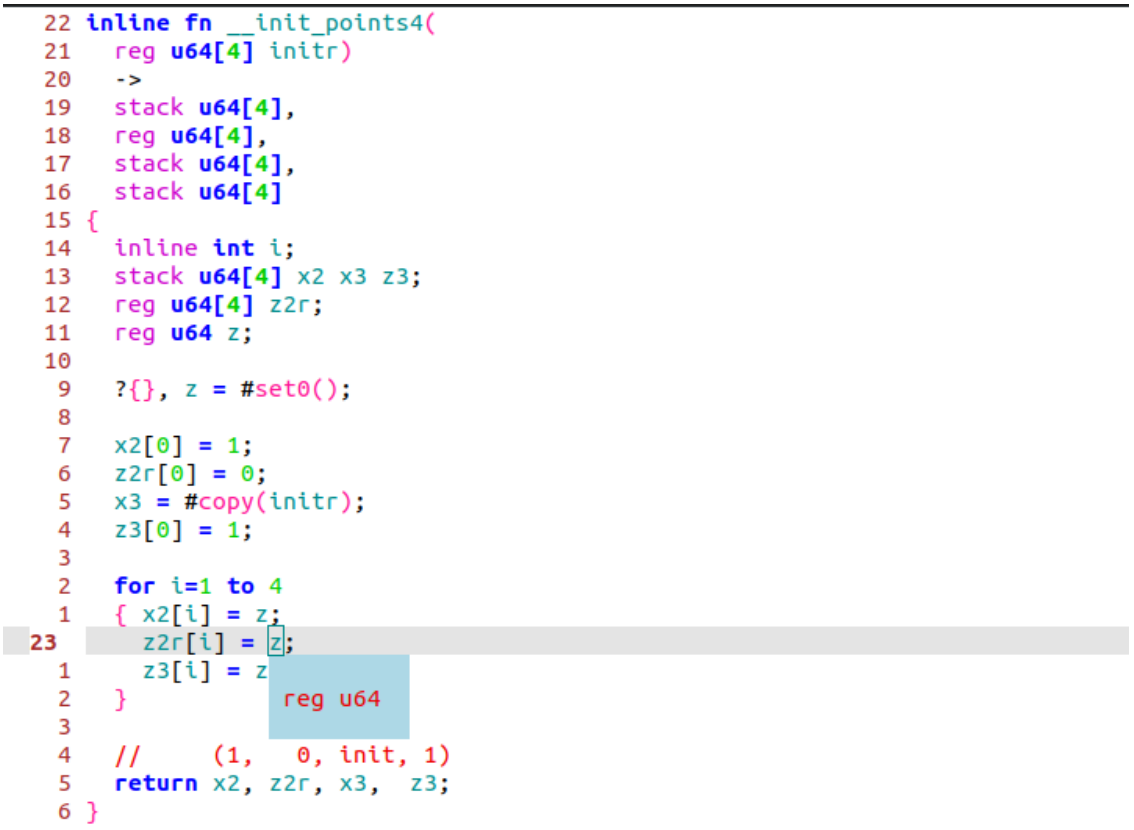

Fig. 4.4. Example of symbol type inference on hover

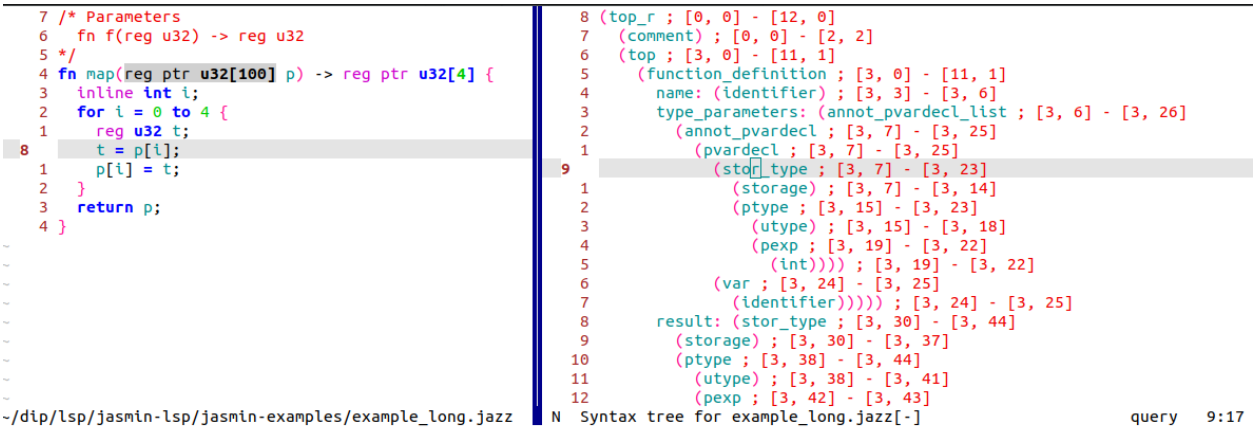

Fig. 4.5. Tree-sitter highlight example

Chapter 5

Evaluation and discussion

In this chapter, we present the results achieved through this work and provide a comparative analysis of our Jasmin language server implementation against the existing implementation, focusing on the features each supports.

5.1 Evaluation

While the language server is still under active development, it implements core functionality that is present in other language server, and it covers basic use-cases and requirements posed in Section 3.1.1.

Table 5.1 below illustrates the feature comparison between the existing Jasmin language server and our implementation. This comparison highlights the enhancements and capabilities we have introduced, showcasing the strides made towards improving the Jasmin development environment.

TABLE 5.1
Feature comparison

Feature Compatibility	Existing Implementation	Our Work
Diagnostics Support	Yes	Yes
Goto Definition Support	No	Yes
Incremental Parsing	Yes	No
Symbol Highlight Support	Yes	Yes
Syntax/Semantic Highlight	No	Syntax Highlight Only
Symbol Type on Hover	No	Yes

Our language server implementation surpasses the existing solution by incorporating additional features. Both versions offer essential diagnostics support; however, we have expanded functionality with the 'goto definition' feature, making code navigation more straightforward for developers.

Additionally, our server displays symbol types on hover, which allows the developers to easily get the symbol type.

While our implementation currently supports only syntax highlighting, it sets the stage for future semantic highlighting enhancements, which will further improve code analysis and editing capabilities.

5.2 Discussion and reflection

Speaking of the challenges faced during the development of the Jasmin language server, one significant issue was the niche status of the Jasmin language itself. Being a relatively specialized and rapidly evolving programming language, the Jasmin compiler presented several undocumented features. This lack of comprehensive documentation posed considerable difficulties in understanding and

integrating these features into the language server.

Another challenge during the implementation was the poor choice of testing methodology, in future releases of the language server we expect to add end-to-end tests, for this the GitHub actions may suite well.

Chapter 6

Conclusion

6.1 Summary

We have designed and implemented the language server prototype, as well as accompanying parsing utility for the Jasmin programming language. Some parts of this work we presented to the group of cryptography researchers, who use Jasmin for their work. This helped us to collect valuable feedback on how to move forward and improve our work.

6.2 Future work

The current implementation of the Jasmin language server is in its early stages and requires further enhancements. For instance, features like context-aware code completion and renaming support need to be developed to improve the programming experience.

Additionally, as discussed in Section 4.1.2, the current system does not support incremental parsing. Implementing this feature requires developing a custom front-

end for the Jasmin compiler, allowing the language server to handle small code changes more efficiently.

Moreover, the implementation of code formatting capability, along with customizable formatting options configuration, would increase readability of the large codebases.

Bibliography cited

[1] Microsoft, *Official page for language server protocol*, 2016. [Online]. Available: https://microsoft.github.io/language-server-protocol/.

[2] *Json-rpc 2.0 specification*, 2010. [Online]. Available: https://www.jsonrpc.org/specification.

[3] Blackduck Inc, *The heartbleed bug*, https://heartbleed.com/, 2014.

[4] J. B. Almeida, J. B. Almeida, M. Barbosa, *et al.*, "Jasmin: High-assurance and high-speed cryptography," *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017. DOI: 10.1145/3133956.3134078.

[5] J. B. Almeida, C. Baritel-Ruet, M. Barbosa, *et al.*, "Machine-checked proofs for cryptographic standards: Indifferentiability of sponge and secure high-assurance implementations of sha-3," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '19, London, United Kingdom: Association for Computing Machinery, 2019, pp. 1607–1622, ISBN: 9781450367479. DOI: 10.1145/3319535.3363211.

[6] G. Barthe, G. Barthe, F. Dupressoir, *et al.*, "Easycrypt: A tutorial," 2013. DOI: 10.1007/978-3-319-10082-1_6.

[7] OCaml LSP development team, *OCaml Language Server Protocol implementation*, version 1.17.0. [Online]. Available: https://ocaml.org/p/lsp/1.17.0.

[8] M. Dénès, *Jasmin language server*, version 0.0.1. [Online]. Available: https://github.com/maximdenes/jasmin-language-server.

[9] F. Bour, T. Refis, and G. Scherer, "Merlin: A language server for ocaml (experience report)," *Proc. ACM Program. Lang.*, vol. 2, no. ICFP, Jul. 2018. DOI: 10.1145/3236798.

[10] Merlin development team, *Merlin*, version 5.4.1, 2025. [Online]. Available: https://github.com/ocaml/merlin.

[11] Semgrep development team, *Semgrep*, version 1.122.0, 2025. [Online]. Available: https://github.com/semgrep/semgrep/tree/develop/src/lsp.

[12] J. Vouillon and J. Dimino, *lwt, Promises and event-driven I/O*, version 5.9.1. [Online]. Available: https://opam.ocaml.org/packages/lwt/.

[13] OCaml community, *yojson*, version 2.2.2, 2024. [Online]. Available: https://github.com/ocaml-community/yojson.

[14] M. Brunsfeld, A. Qureshi, A. Hlynskyi, *et al.*, *Tree-sitter/tree-sitter: V0.25.5*, version v0.25.5, May 2025. DOI: 10.5281/zenodo.15531772.

[15] F. Pottier and Y. Régis-Gianas, *Menhir, parser generator for OCaml*. [Online]. Available: https://gallium.inria.fr/~fpottier/menhir/.

[16] Libjade development team, *libjade, Crypto library*. [Online]. Available: https://github.com/formosa-crypto/libjade.